GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 05 — Sockets TCP y UDP
Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Lo que veremos en este bloque



UDP
sendto · recvfrom · datagramas



HTTP
El protocolo que mueve la web



NTP
La hora de Internet sobre UDP



¿TCP o UDP?
La regla de decisión



Práctica eco
Dos servidores, dos protocolos

Partimos de lo construido en el bloque 01: `socket`, cliente y servidor TCP, handshake, `SO_REUSEADDR` y timeouts.

Cliente y servidor UDP

La idea en una frase

> UDP **no se conecta**: el cliente manda un datagrama con `sendto()` y espera con `recvfrom()`; el servidor solo hace `bind()` y escucha. **Sin connect(), sin accept(), sin listen()**... y sin garantía de que el mensaje llegue.

Aquí compruebas en la API las tres filas de «No» de la tabla del bloque 01: construimos el cliente y el servidor UDP, y aterrizamos las consecuencias de lanzar aviones de papel.

El cliente UDP

sendto() y recvfrom()

cliente_udp.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 5001
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(b"Hola UDP!", (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

Línea	Qué hace
`socket(AF_INET, SOCK_DGRAM)`	Crea el socket UDP (`SOCK_DGRAM`)
`sendto(b"Hola UDP!", (HOST, PORT))`	Envía el datagrama: **la dirección viaja en la llamada**
`recvfrom(1024)`	Devuelve la tupla `(datos, dirección)` ; aquí la dirección se descarta con `_`

Sendto contra send

La diferencia visual con TCP

Cliente TCP (bloque 01)	Cliente UDP (aquí)
`s.connect((HOST, PORT))`	— (no existe)
`s.sendall(datos)`	`s.sendto(datos, (HOST, PORT))`
`s.recv(1024)`	`s.recvfrom(1024)`

ℹ️ En TCP primero te conectas y luego envías; en UDP el «a quién» va **dentro del envío**.

⚠️ ⚠️ Cada `sendto()` es un datagrama independiente: pueden llegar **desordenados, duplicados o no llegar**. Si el servidor no escucha, el datagrama se pierde y `recvfrom()` espera **para siempre** (salvo `settimeout()`).

El servidor UDP

bind() y recvfrom(): nada más

servidor_udp.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 5001
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor UDP en {HOST}:{PORT}")
    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")
    servidor.sendto(b"Recibido!", direccion)
```

1. `bind()` reserva el puerto: todo datagrama que llegue a 5001 es para este proceso
2. `recvfrom()` bloquea hasta que llega un datagrama y devuelve **contenido + dirección**
3. `sendto(respuesta, direccion)`: la única forma de «devolver la pelota», porque no hay conexión que recuerde quién eras

✅ Consecuencia: un solo servidor UDP atiende a **cualquier número de clientes** sin accept() ni hilos.

Sin accept(), sin listen()

Servidor TCP contra servidor UDP

Servidor TCP (bloque 01)	Servidor UDP (aquí)
`bind()`	`bind()`
`listen()`	— (no existe)
`accept()` → devuelve `conn`	— (no existe)
`conn.recv(1024)`	`servidor.recvfrom(1024)`
La conexión sabe quién es el cliente	`recvfrom()` devuelve la dirección

ℹ️ 💡 En localhost (127.0.0.1) los datagramas casi nunca se pierden ni desordenan: loopback, misma máquina. El desorden y la pérdida se ven en **redes reales**, con routers y congestión.

Datagramas: cada envío es un mundo aparte

cinco_aviones.py — Python

```python
import socket, time
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    for i in range(1, 6):
        cliente.sendto(f"Mensaje #{i}".encode(), ("127.0.0.1", 5001))
        time.sleep(0.2)
```

⚠️ **Los males de UDP:**

- **Pérdida**: sin confirmación, si un paquete se pierde, se pierde para siempre
- **Desorden**: sin números de secuencia, nadie reordena los datagramas
- **Duplicados**: el mismo datagrama puede llegar dos veces, sin detección
- **Tamaño**: máx. 65.535 − 8 (UDP) − 20 (IPv4) ≈ **65.507 bytes**; más → `OSError`

ℹ️ ¿Por qué alguien elegiría esto? Porque esos males son el **precio de la velocidad**: sin handshake ni reenvíos, cada paquete cuesta el mínimo.

Mini-chequeo: UDP

Responde en 30 segundos:

1. ¿Qué dos métodos usa el cliente UDP y qué devuelve cada uno?
2. ¿Por qué el servidor UDP no necesita `accept()`?
3. ¿Qué tres cosas pueden pasarle a un datagrama en la red y qué pasa si se pierde?
4. ¿Cuál es el tamaño máximo de un datagrama UDP?

Respuestas:

- `sendto(datos, dirección)` envía; `recvfrom(1024)` devuelve **(datos, dirección)**
- Porque UDP **no tiene conexiones**: cada datagrama llega independiente y `recvfrom()` dice de quién viene
- Puede **perderse**, **llegar desordenado** o **duplicarse**; si se pierde, para siempre
- Unos **65.507 bytes** de carga útil (65.535 menos cabeceras UDP e IPv4)

HTTP: el protocolo que mueve la web

> TCP y UDP son solo la **tubería**; los protocolos de aplicación deciden **qué** se escribe dentro. **HTTP** es un protocolo de **texto sobre TCP**: petición y respuesta son texto claro.

petición GET — HTTP

```
GET /ruta HTTP/1.1\r\n
Host: ejemplo.com\r\n
User-Agent: MiNavegador\r\n
Accept: text/html\r\n
\r\n
```

La línea en blanco `\r\n\r\n` marca el final de las cabeceras: «ya estaba todo». `Host:` es obligatoria en HTTP/1.1.

Verbos y respuesta HTTP

Verbo	Qué pide
GET	Leer un recurso
POST	Enviar datos (un formulario)
PUT	Guardar/reemplazar un recurso
DELETE	Borrar un recurso

respuesta HTTP

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 1234\r\n
\r\n
<html>...cuerpo...</html>
```

ℹ️ **200** todo bien; también verás **404 Not Found** y **500 Internal Server Error**. `Content-Length` dice cuántos bytes mide el cuerpo: así sabes cuándo se acabó.

El cliente HTTP manual

http_manual.py — Python

```python
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("www.example.com", 80))
    s.sendall(b"GET / HTTP/1.1\r\nHost: www.example.com\r\n"
              b"Connection: close\r\n\r\n")
    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos
print(respuesta.decode()[:500])
```

1 `Connection: close`: HTTP/1.1 mantiene la conexión abierta (*keep-alive*); así forzas al servidor a cerrarla al terminar

2 La respuesta llega **en trozos**: el bucle acumula bytes hasta que `recv()` devuelve `b""`

El orden de los bytes (network byte order)

> Cuando envías números por la red, sus bytes van en un orden acordado: **big-endian**, también llamado **network byte order**. En Python se fuerza con el prefijo `!` de `struct`.

byte_order.py — Python

```python
import struct
# '!' = big-endian
datos = struct.pack("!I", 5000)
print(datos)            # b'\x00\x00\x13\x88'
puerto = struct.unpack("!I", datos)[0]
print(puerto)           # 5000
```

Código	Significado
`!`	Network byte order (big-endian)
`I`	Entero sin signo de 4 bytes
`H`	Entero sin signo de 2 bytes

ℹ️ `!I` y `!H` aparecen en todos los protocolos que hablan números. Es el truco que necesitas para el NTP que viene ahora.

NTP: sincronizar relojes con UDP

El reloj de Internet

1. Tu ordenador manda un datagrama UDP al **puerto 123** de `pool.ntp.org`, un grupo mundial de servidores de tiempo
2. NTP manda **múltiples peticiones** y cruza respuestas para descontar la latencia
3. Un datagrama perdido es **irrelevante**: se promedia estadísticamente. Por eso puede permitirse UDP

hora_ntp.py — Python

```python
import socket, struct, time
def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        # 48 bytes, modo cliente
        paquete = b"\x1b" + 47 * b"\0"
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)
    # Timestamp en los bytes 40-43
    t = struct.unpack("!I", datos[40:44])[0]
    # Época NTP (1900) → Unix (1970)
    return t - 2208988800
print(time.ctime(hora_ntp()))
```

Cómo funciona el cliente NTP

Tres detalles que importan

1. `b"\x1b" + 47*b"\0"`: el primer byte `00011011` activa el **modo cliente** y la versión 3; los 47 bytes restantes son relleno del formato de 48 bytes
2. `settimeout(5)`: imprescindible en UDP; si la respuesta no llega, excepción en vez de bloqueo eterno
3. `struct.unpack("!I", datos[40:44])`: el servidor escribe su hora como entero de 4 bytes en **network byte order** en las posiciones 40-43

ℹ️ **SNTP** (Simple NTP) es la versión ligera para routers, móviles e IoT: para el 99% de las aplicaciones, más que suficiente.

Mini-chequeo: HTTP, bytes y NTP

🧠 Responde sin mirar:

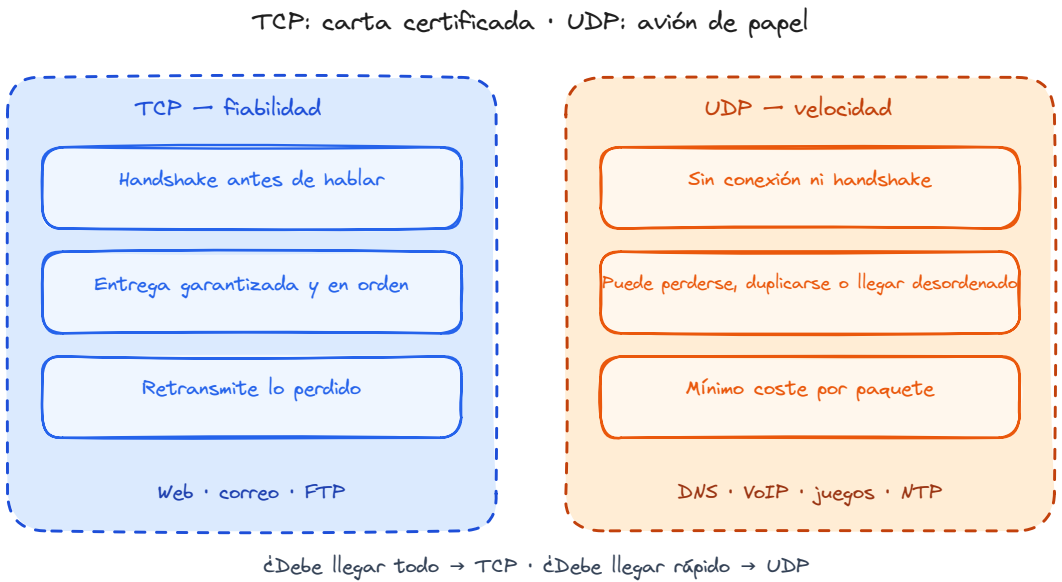
1. ¿Qué significa la línea en blanco `\r\n\r\n` en una petición HTTP y qué indica el código 200?
2. ¿Qué código de `struct` usa big-endian y por qué lo necesita el cliente NTP?
3. ¿Por qué NTP usa UDP si la hora debe ser exacta?

🔄 **Respuestas:**

- Que la petición **ha terminado** (fin de cabeceras); **200 OK** = procesada correctamente
- El prefijo **!** (ej: `!I`): el servidor NTP escribe su timestamp en network byte order, hay que leerlo igual
- Manda **muchas peticiones** y calcula la hora estadísticamente: un paquete perdido no cambia nada

¿TCP o UDP?

La pregunta correcta



¿Ambas cosas? **QUIC** (la base de HTTP/3) construye fiabilidad tipo TCP **encima de UDP**: confirmaciones, control de congestión y cifrado.

Casos reales

La tabla que se repite en las entrevistas

Aplicación	Protocolo	Por qué
Web (HTTP)	**TCP**	La página debe llegar completa y en orden
Correo (SMTP, POP3, IMAP)	**TCP**	Un mensaje a medias es inútil
Transferencia de archivos (FTP)	**TCP**	Un byte perdido corrompe el archivo
DNS	**UDP**	La consulta es diminuta; si se pierde, se repite
VoIP / videollamadas	**UDP**	Debe ir fluida; un frame perdido se tolera
Juegos online	**UDP**	Cientos de paquetes por segundo; la latencia manda
NTP (hora)	**UDP**	Manda muchas peticiones y promedia

⚠️ Ojo: el *streaming web* moderno (Twitch, YouTube) usa **HTTP sobre TCP** (segmentación HLS/DASH), no UDP: busca llegar a todo el mundo por infraestructura web.

Mini-chequeo: criterio de decisión

🧠 Responde en 30 segundos:

1. ¿Qué criterio usarías para elegir entre TCP y UDP en una app nueva?
2. ¿Por qué DNS usa UDP si «perder» la respuesta de un dominio parece grave?
3. Pon dos ejemplos de TCP y dos de UDP distintos de los de la tabla.

🔄 **Respuestas:**

- ¿Puedes permitirte perder datos? Dato **intacto y completo** → TCP; **fluidez tolerando pérdidas** → UDP
- La consulta es **diminuta** y repetirla cuesta casi nada: la fiabilidad sale de **re-preguntar**
- TCP: descargas de actualizaciones, pago online. UDP: retransmisión en vivo, chat de voz de un juego, telemetría de sensores

Práctica eco

La idea en una frase

> Un **servidor eco** devuelve al cliente exactamente lo que recibe. Lo montas **dos veces** —TCP y UDP— para notar con tus manos la diferencia entre canal fiable y datagramas sueltos.

Juntas todo lo aprendido: cliente y servidor TCP, `SO_REUSEADDR`, timeouts y datagramas. Con **dos terminales** verás cada protocolo completo en acción.

Eco en TCP

eco_tcp.py — Python

```python
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000)); srv.listen()
    print("Servidor eco TCP escuchando en 127.0.0.1:9000")
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        print(f"Recibido de {addr}: {datos.decode()}")
        conn.sendall(datos)  # Eco
```

cliente_tcp.py — Python

```python
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Prueba")
    print(cli.recv(1024).decode())
```

```
$ python cliente_tcp.py
Prueba
```

El truco está en `conn.sendall(datos)` : devuelve los **mismos bytes** que llegaron. Eco, literalmente.

Eco en UDP

eco_udp.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 9001
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor eco UDP en {HOST}:{PORT}")
    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")
    servidor.sendto(datos, direccion)  # Eco
```

cliente_udp.py — Python

```python
import socket
HOST, PORT = "127.0.0.1", 9001
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    mensaje = "Hola UDP, devuélveme esto"
    cliente.sendto(mensaje.encode(), (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"El servidor me devolvió: {datos.decode()}")
```

```
$ python cliente_udp.py
El servidor me devolvió: Hola UDP,
devuélveme esto
```

`sendto(datos, direccion)` devuelve los mismos bytes **a la dirección de la que vinieron**.

Las dos versiones, cara a cara

	Eco TCP (puerto 9000)	Eco UDP (puerto 9001)
El servidor espera con	`accept()` + `recv()`	`recvfrom()`
Devuelve con	`conn.sendall(datos)`	`servidor.sendto(datos, direccion)`
¿Quién es el cliente?	La conexión lo sabe	`recvfrom()` entrega la dirección
¿Varios a la vez?	De uno en uno (o con hilos)	Cualquiera, sin hilos

ℹ️ `accept()`, `recv()` y `recvfrom()` son **bloqueantes**: el programa espera hasta que algo ocurre. El servidor en una terminal, el cliente en otra: cada protocolo completo funcionando.

Aprieta el lápiz

Seis retos sobre lo aprendido

1. **Eco TCP**: servidor que devuelve exactamente lo que recibe
2. **Eco UDP**: lo mismo, con datagramas
3. **Contador de letras**: el cliente envía una frase, el servidor responde cuántas letras tiene
4. **Servidor hora**: el cliente se conecta y recibe la hora actual
5. **Cliente con timeout**: si no hay respuesta en 3s, «Servidor no disponible»
6. **Comparativa velocidad**: ¿cuánto tardan TCP y UDP en enviar 100 mensajes pequeños?

Soluciones de los retos 1-3

Eco TCP, eco UDP y contador

retos_1_2_3.py — Python

```
# 1. Eco TCP
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000)); srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos)
#
# 2. Eco UDP
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    srv.sendto(datos, direccion)
#
# 3. Contador de letras (dentro del with conn)
```

Soluciones de los retos 4-6

Hora, timeout y comparativa

retos_4_5_6.py — Python

```
# 4. Servidor hora (dentro del with conn)
    hora = time.strftime("%H:%M:%S")
    conn.sendall(hora.encode())
#
# 5. Cliente con timeout
with socket.socket() as cli:
    cli.settimeout(3)
    try:
        cli.connect(("127.0.0.1", 9000))
        cli.sendall(b"hola")
        print(cli.recv(1024).decode())
    except (socket.timeout, ConnectionRefusedError):
        print("Servidor no disponible")
#
# 6. Velocidad: cronometrar 100 envíos TCP vs UDP
#    con test_tcp() y test_udp() por separado
```

Sé el socket

Consolida lo aprendido

1. `bind(("127.0.0.1", 9000))` : reservas el puerto. **Ya eres la centralita**
2. `listen()` : el SO ya sabe que esperas llamadas. Te quedas en `accept()` ... **esperas**
3. Un cliente hace `connect()` : el handshake (SYN → SYN+ACK → ACK) suena como el timbre. `accept()` despierta y devuelve el **nuevo socket** de esa conversación
4. `recv(1024)` : **esperas otra vez**. Cuando el cliente manda `b"Hola!"` , despiertas y contestas con `sendall()`
5. Al salir del `with` , la despedida FIN/ACK cierra la conversación... y tú vuelves a tu `accept()` : **esperando al siguiente cliente**

ℹ️ ¿Y si otro cliente llama mientras atiendes a uno? Queda en la **cola** del `listen()` (el *backlog*): un solo hilo atiende a un cliente cada vez.

¿Quién soy?

Adivina el concepto

1. Soy el punto final de una conexión de red
2. Soy el método del cliente TCP que estrecha la mano antes de hablar
3. Soy el método del servidor UDP que dice de quién vienen los datagramas
4. Soy el estado que retiene el puerto tras cerrar
5. Soy la opción que evita el «Address already in use»
6. Soy el protocolo de texto sobre TCP que mueve la web
7. Soy el grupo de servidores de tiempo del puerto 123
8. Soy el protocolo fiable sobre UDP de HTTP/3
9. Soy la llamada que espera en varios sockets a la vez sin hilos
10. Soy la tupla que devuelve `recvfrom()`

🔄 **Respuestas:** 1. El socket · 2. `connect()` · 3. `recvfrom()` · 4. TIME_WAIT · 5. `SO_REUSEADDR` · 6. HTTP · 7. `pool.ntp.org` (NTP) · 8. QUIC · 9. `select()` · 10. `(datos, direccion)`

Entrevista de trabajo

Las seis preguntas que te harán

1. ¿Qué diferencia hay entre TCP y UDP? ¿Cuándo usarías cada uno?
2. Escribe un servidor TCP que reciba un mensaje y lo devuelva
3. ¿Cómo funciona el three-way handshake? Explícalo con un diagrama
4. ¿Qué errores pueden ocurrir al comunicar por sockets y cómo los gestionas?
5. ¿Qué es SO_REUSEADDR y cuándo lo necesitas?
6. ¿Cómo funciona HTTP a nivel de socket? Descríbeme una petición y su respuesta

✅ 💡 La 1 y la 2 son las «preguntas reina»: para la 1, la moraleja **fiabilidad contra velocidad** con los casos reales; para la 2, escribe el servidor sin pensarlo: `socket() + bind() + listen() + accept() + recv() + sendall()`. Si lo cuentas fluido, ya eres medio desarrollador de redes.

Resumen del bloque 02

✈️ **UDP**
sendto/recvfrom; sin accept; datagramas de ~65.507 B

🌐 **HTTP**
Texto sobre TCP; verbos y códigos; recv en bucle

🔢 **Bytes**
Network byte order con struct "!

⏰ **NTP**
48 bytes al puerto 123; promedia; settimeout

🛠️ **Eco**
TCP 9000 · UDP 9001; dos terminales

✅ Siguiente parada: **UD 06 · Servidores concurrentes** — hilos y `select()` para atender a muchos clientes a la vez.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //